ASSIGNMENT 8 · OPTIMIZATION TECHNIQUES LAB

Optimize Anything

Exam timetabling as the main case study, plus nine more tracks: compilers, xv6, PostgreSQL, quantum, parallel and test suites

Akhil · November 2026 · built by prompting Claude, every use disclosed

The problem: placing every exam in a slot

Two exams sharing even one student can’t share a slot. Students also need gaps between their exams.

The usual manual method walks the course list and drops each exam into the first free slot.

1 of 7

real institutions (Toronto benchmark) where first fit finds a clash-free timetable at all

Source: Track E, results/SUMMARY.md · Toronto data from Carter, Laporte and Lee (1996)

Formulation

VARIABLES

$x_{et} \in \{0, 1\}$

1 if exam e is held in slot t

HARD CONSTRAINTS

Each exam in exactly one slot

No two conflicting exams in the same slot

OBJECTIVE

Minimise the Carter proximity cost: each shared student costs 16 one slot apart, 1 five apart, 0 beyond

Binary variables and an NP-hard core (graph colouring) rule out gradient methods. The plan: greedy construction and local search at scale, ILP for small cases, bounds and repairs.

Methods compared

Method	Chapter 6 family	Role
First fit by course id	greedy	baseline
Largest degree, DSatur	greedy	better orderings
DSatur-proximity	greedy	cheapest legal slot
+ hill climbing	local search	optimized method
Min-slots ILP	ILP, branch-and-bound	feasibility, colouring
Proximity ILP, warm-started	ILP, branch-and-bound	small instances

Real data: seven institutions

Instance (exams / slots)	First fit (baseline)	DSatur	DSatur-proximity + hill climb
ear-f-83 (190 / 24)	infeasible	67.38	40.83 ± 0.24
hec-s-92 (81 / 18)	infeasible	infeasible	15.79 ± 1.04 (with ILP repair)
lse-f-91 (381 / 18)	infeasible	infeasible	13.00 ± 0.04
sta-f-83 (139 / 13)	211.93	194.40	168.95 ± 0.29
tre-s-92 (261 / 23)	infeasible	15.92	10.09
ute-s-92 (184 / 10)	infeasible	58.82	31.70 ± 0.31
yor-f-83 (181 / 21)	infeasible	61.66	46.30 ± 0.06

Proximity cost per student, lower is better; 5 seeds; under 0.2 s per run. Best known: tre 7.75, yor 34.84, so ours are 30–33% higher.

When greedy fails: exact for feasibility, heuristic for cost

Every greedy method

hec-s-92 has no clash-free timetable in 18 slots

→

Min-slots ILP, 20 s

Clash-free in 18 slots, cost 25.25

→

Hill climbing

Cost 15.79 ± 1.04 (best 14.27)

Exact methods prove and repair; heuristics scale. The proximity ILP never proved optimality: its LP bound stays at 0.

Nine more tracks, one number each

F · Compiler

4.89× fewer cycles, 0 miscompiles in 300 programs

G · xv6

MLFQ −56% interactive turnaround; COW fork 14× faster

H · PostgreSQL

TPC-H 51.6 s → 1.5 s with ILP-chosen indexes, same results

J · Test suites

17 s → 2.1 s of test time, full coverage kept

D · Parallel

3.40× on 4 cores; greedy hits the coverage optimum

A · Continuous

κ = 1,000: GD 2,695 iterations, Nesterov 138, Newton 1

B · Discrete

Greedy within 0.01–3% of exact knapsack

C · QAOA (simulated)

0.93 approximation ratio; local search 0.95

I · Grover (simulated)

50 vs 2,019 queries; 38 s vs 89 µs wall time

Measured: D, F, G, H, J · Simulated: C, I · Every track has tests, a baseline and a results/SUMMARY.md

Where optimizing didn’t pay

COW fork, child writes everything

2.3× slower: 1,024 page faults instead of one bulk copy

QML vs classical

Below the best classical model on both datasets; 15–83 s to train in simulation vs under 0.3 s

Test-suite minimization alone

61% fewer tests but only 15% less time; 7–11% of seeded faults missed

Memory-bound parallel work

Slower than serial on the 4-core VM: more cores, no more bandwidth

Profiling first found the real bottlenecks: two tests held 73% of test time, two TPC-H queries 97% of query time.

Vibe coding, checked

16

AI-generated errors found and fixed, each in the failure log

An ILP reporting 4× its real cost; a fuzzer blind to bugs; a test set from a different task; strided shards that undid data parallelism.

Caught by independent checks: evaluators, brute force, differential testing, mutation testing, a second machine.

Tool comparison: Claude Sonnet 5.5 and Haiku 5.5 got the same CSE prompt; both 10/10 correct, Sonnet twice as fast.

Key lessons

1. Measure the baseline and profile before optimizing.
2. Check assumptions before quoting guarantees: greedy’s 63% bound failed where submodularity did.
3. Exact methods prove and repair; heuristics scale. Combine them.
4. Estimates mislead: planner costs, index sizes, simulated quantum speed-ups.
5. AI writes fast; independent checks make it correct.